Cloud Computing Mini Project: Distilled Command Parser on Kubernetes

Thirumalasetty Karthik | Roll 230101007 | Repository: https://github.com/SudeepKarthik30/distilled-command-parser-k8s | Model release: v1.0

1. Overview

A Phi-3.5 teacher model (served locally via Ollama) annotates natural language voice commands with structured intent and slot boundaries. A compact Pythia-70M student is fine-tuned on the verified teacher annotations via knowledge distillation. The student (v1) is served through an asynchronous FastAPI microservice (/healthz, /readyz, /parse) deployed in a k3d Kubernetes cluster backed by SeaweedFS S3-compatible object storage. Model weights are published as GitHub Release v1.0 and decoupled from version control. The full stack is fully reproducible from a clean clone via make up.

2. Data and cleaning

3,000 teacher-annotated examples were audited and cleaned using worker/clean.py. The cleaning pipeline rejected 4 malformed JSON strings, 7 invalid key sets, 0 out-of-domain intents, 81 hallucinated/unmatched slot spans, and 48 duplicate query commands (140 rejected records total), leaving 2,860 verified high-quality training pairs.

3. Model quality

Model evaluation was conducted over 100 gold validation benchmarks (results/score_*.txt).

Model Architecture	JSON Valid	Intent	Date	Time	Place	Full Frame
Untrained Pythia-70M	0%	0%	0%	0%	0%	0%
Teacher (Phi-3.5)	82%	59%	65%	76%	71%	39%
Student v1 (Deployed)	99%	75%	69%	82%	57%	19%
Student v2 (Greedy)	99%	75%	69%	82%	57%	19%

The untrained baseline yielded 0% across all metrics because it generated repetitive token sequences failing the required schema. Post-distillation, Student v1 achieved 99% JSON validity, outperforming the teacher model on JSON compliance (+17%), intent classification (+12%), and time slot extraction (+14%). The teacher retained higher full-frame accuracy (39% vs 29%) due to simultaneous multi-slot fidelity. A repetition penalty of 1.1 degraded decoding into truncated structures; hence greedy unpenalized decoding was retained for production serving.

4. Deployment and load test

Performance benchmarks were gathered using headless Locust across 4 concurrent client workers over 2 minutes against 2 active replicas.

Evaluation Metric	Observed Value
Requests / Failures	342 / 2 (0.58% failure rate)
Latency (p50 / p95)	1,250 ms / 2,700 ms
Throughput	2.85 requests / second
CPU Utilization per Pod	1928m and 1915m (of 2000m ceiling)
Memory Consumption per Pod	742 MiB and 758 MiB (of 1 GiB ceiling)
Traffic Distribution Across Replicas	172 reqs / 168 reqs (50.6% / 49.4% split)
Cold-Start Readiness (Median of 3 runs)	7.1 s (run measurements: 8.2 s, 6.8 s, 7.1 s)

5. Recovery and autoscaling

Pod resilience was validated under sustained load by intentionally terminating an active Pod. Out of 450 requests transmitted during the failover event, only 4 returned errors (0.89% failure rate, all transient HTTP 500), and the replacement Pod achieved Ready status within 17.5 seconds. Horizontal Pod Autoscaler (HPA target CPU: 70%, replica span: 1 to 2) triggered scale-up from 1 to 2 replicas when aggregate CPU exceeded 195%, and smoothly scaled back down to 1 replica approximately 5 minutes after load cessation.

6. Cost and SLO

Cost per replica-hour is calculated via 2 vCPU x 3.50 INR + 1 GiB x 0.45 INR = 7.45 INR/h, totaling 14.90 INR/h for a dual-replica deployment. At an observed throughput of 2.85 req/s (10,260 requests/hour), the unit economic cost is 1.45 INR per 1,000 processed requests. The service level objective (SLO) requires >= 99% successful requests per test run. The SLO was cleanly satisfied in the standard baseline run (99.42% success; 2 errors against a 3.42 error budget) and during simulated pod kill (99.11% success).

7. Component analysis

Component	Identified Limitation	Recommended Engineering Improvement
Data & Object Storage	Single-node SeaweedFS without HA replication; cold starts fetch weights over cluster network at every pod launch.	Implement distributed multi-node storage backend; attach persistent volume caches (PVC) directly to nodes.
Model Quality	Compact 70M parameter student exhibits occasional syntax errors and slot span boundary hallucinations.	Incorporate larger distillation pools, synthetic edge cases, and grammar-constrained JSON decoding (e.g. Outlines/Guidance).
Latency & Throughput	Inference is CPU-bound; thread contention occurred prior to setting OMP/MKL thread clamps (p95 was 7.7s).	Leverage INT8/INT4 quantization (bitsandbytes/ONNX Runtime) and dynamic continuous batching.
Resource Utilization	Pods saturated near 100% CPU limits at only 4 concurrent users.	Optimize inference runtime, right-size memory limits, and horizontal scale across multi-node topology.
Reliability & HA	Local single-node k3d environment lacks true physical host redundancy.	Multi-node Kubernetes cluster with node anti-affinity, pod disruption budgets (PDB), and higher HPA max limits.
Security & Governance	Storage credentials stored in base64 Secrets; Ingress operates over unencrypted HTTP.	Integrate HashiCorp Vault / External Secrets Operator, cert-manager for automated TLS termination, and mTLS.

8. Use of AI

Antigravity (Gemini) and Anthropic Claude were utilized for architecture planning, manifest drafting, and diagnosing PyTorch thread oversubscription under CPU limits. Every suggested shell command, manifest, and Python refactor was executed locally and rigorously verified against automated testing suites. The stateless NACL was tested and verified with tools/nacl_sim.py (6/6 passing, 100%). Model scoring was audited with tools/score.py, and request timings were measured directly via Locust. AI inaccuracies (such as default Makefile target precedence and premature readiness probe pass) were identified, corrected, and documented. Distillation labels were produced exclusively by Phi-3.5 via Ollama without synthetic manual fabrication. Full verification logs are detailed in AI_USE.md.